

Weight Regression for federated setting knowledge combination

Aymen Tlili

September 2024

phD student position Application - Project proposition

To the esteemed professors of Università di Pisa



1 Introductive Summary

The Federated Learning setting supposes the aggregation of models of differing properties and tasks where they "splash"/share knowledge with each other by acting on their weights and building similarities. This resembles EWC in continual learning and Our proposal aims to skip training/finetuning steps to reach workable models while constrained to little to no Dataset access and operating on already trained models instead.

Predicting a set of weights given 2 flattened model parameters sampled from model ZOOs as input that is expected to be functional for inference would guarantee privacy concerns being mitigated but also saves us memory and compute power. We seek to find topological properties of high performing models on their respective real image classes, relationships in the loss surface allowing us to skip its traversal and disentangle the role of each neurone, making A.I more explainable and allowing us to build upon works that track bifurcation of weights to find which interactions are responsible for performance on said Class. The novelty in our approach can be seen as :

- the functional 2-branch-input for such a task, building upon models in competition with diffusion models and multi-head attention autoencoders for regression/reconstruction.

- Creating a new Dataset Model zoo Benchmark of all possible Class Combinations same-architecture-CNNs of the 10 class MNIST dataset and rearranging those models into pairs to feed our Autoencoder for regression taken from different activation functions and sampled after maturity every 5 epochs.
- Finding a loss function that works on long sequences and able to preserve both the shape(Topological manifold) and content of data(observable information and inference performance) for a repurposed AutoEncoder . The direction that current work will be displayed sees sinkhorn/Gromov-Wasserstein distance and Persistent homology and the Fischer information metrics as prime candidates with justified motives to optimize the knowledge combination on .
- We investigate spectral evolution of weights to discern which are responsible for generalization and which are still flexible enough to learn new tasks, As sparsification and model Knowledge distillation have showed that 5-20% of weights are all what you need from a network to perform adequately on out-of-training-distribution scenarios.effectively attempting to extend works that aim to replicate grokking in language models .
- At worst , if we have to relax the federated learning constraints and use less than 1%-sized samples of the original image Datasets to finetune our datasets , we propose to still preserve privacy by applying dataset distillation methods on said samples or ask for it as a requirement from federated learning participants.

2 Survey of Background Literature

2.1 Overview[non-technical]

As we march into an era where A.I becomes more accessible and many people confuse it's totality with Language Models or incite discontent among artists being replaced by diffusion models , Data Efficiency and Privacy are 2 major cornerstones for modern intelligent systems and they are not to be sacrificed for performance when wielding Deep learning models ,which the bigger they are , the harder they are to make sense of . Thus raising distrust both about the Ethical/compute efficient side of training stage resources and the inference stage trustworthiness if even the experts that set it in place don't have full omniscience over it's interactions.

We attempt to take on a multi-faceted such problem with the tools at hand.If deep learning models are universal function approximators that can be trained to replace any operand , why not Train one to be the "Union" or "addition" operator for said models ? Could we even approximate what a training step is ,as an operator, with successive calls to the same model ?

Using the A.I that we can understand to make sense of the A.I we don't could be the next step to smarter models and building blocks to more efficient compute .

A.I Experimenters , Robotics companies , Pharmaceuticals companies,Genetic engineers,Developers,Physicists , Artists ,Data Analysts ... would greatly benefit if we understood more and could use these impactful tools at will and eventually understand them enough to customize them and could benefit from the knowledge of other models in making our own with no fear of private data leaking or ruining lives in the process.

2.2 Background

Previous works operating on Models would be enlisted under AutoML and the manipulation of hyper-representation .AutoML contains many sub-branches that each focus on predicting / inferring certain properties about neural networks .

Neural Architectural Search(NAS) which is an automated process for designing neural network architectures. Instead of manually crafting models, NAS explores a search space of possible architectures to identify the optimal one for a given task. It uses techniques like reinforcement learning, evolutionary algorithms, or gradient-based methods to optimize performance metrics, such as accuracy or efficiency. NAS aims to find architectures that outperform human-designed networks while reducing the time and expertise required to design them. It's widely used in applications like image recognition, natural language processing, and more.

Graph Neural Networks (GNNs) can be used to predict optimal hyperparameters, such as batch size and learning rate, by modeling the relationships between different training setups. GNNs represent hyperparameter configurations as nodes in a graph, with edges capturing their similarities or transitions. By learning from previous training outcomes, GNNs predict the impact of varying hyperparameters on model performance. This enables the discovery of promising hyperparameter values more efficiently than exhaustive grid searches or random sampling. GNN-based approaches leverage the structural properties of hyperparameter space, making optimization faster and more scalable, especially for complex deep learning models.

Kaggle competitions sometimes focus on meta-learning tasks, such as predicting training time required to achieve a specific performance metric, if possible. In these contests, participants build models that forecast how quickly a machine learning model will reach a desired accuracy or loss, given the dataset, architecture, and hyperparameters. These predictions help optimize resource allocation and reduce experimentation time. Competitions of this nature push innovation in performance prediction and autoML, benefiting real-world applications where training time and compute cost are critical factors.

But Scarcely do we ever see the weight generation problem taken head on or analysed to great depth , leaving us , even some researchers , using deep learning as an expensive black box toy.We can break down the levels of data Privacy /compute efficiency/interpretability barriers as such :

1. infer a model that has 10% less parameters which are fully explainable and train/finetune on future tasks 5% of distilled privacy-and-performance-preserving data [what we want to help achieve]
2. Weight Regression for federated setting knowledge combination [Ongoing work and is The proposed approach]
3. We have access to the training data or an embedding/encoded version of it but we apply our transformative model. so a sequence of steps would be :
 - Flatten a model's weights ; save an embedded version it worked on
 - Feed the weights to an Attention-based Autoencoder(Reconstruction task) or a Diffusion model (Regression task with inference conditioned of the embedded dataset) . Both outputting 2%-15% accuracy performance predicted weights
 - Finetune predicted weights with the Data we kept aside in first step to reach 95%+ reconstruction rate and performant models to deploy saving hours of compute and replacing it with a forward pass from their model

[Works like "Diffusion-based Neural Network Weights Generation" and "Hyper-representations as generative models: Sampling unseen neural network weights." and "Graph hypernetworks for neural architecture search"]

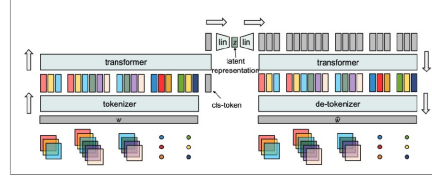


Figure 1: Hyper-representations as generative models: Sampling unseen neural network weights proposed architecture with the strategy of sampling latent representations and decoding them

Table 9: Performance of the datasets conditional sampling on 10 unseen real-world datasets. We report the averaged accuracy on ten unseen test datasets over 3 different runs fine-tuned for 50 epochs. pret(imnet): pretrained on imagenet1k

Datasets	Random init.	No-fine-tuning pret(imnet)	D2NWG(ours)	Random init.	50 epochs Fine-Tuning pret(imnet)	D2NWG(ours)	# of classes
Gemstones	1.13 ± 0.52	0.62 ± 0.00	1.86 ± 0.25	70.59 ± 0.91	67.49 ± 0.43	76.06 ± 0.88	87
Dog Breeds	0.55 ± 0.22	0.69 ± 0.00	1.87 ± 0.39	80.78 ± 0.28	78.13 ± 0.49	80.88 ± 0.88	133
Dessert	21.03 ± 2.44	12.50 ± 0.00	99.40 ± 0.02	95.83 ± 0.34	94.64 ± 0.00	99.40 ± 0.02	5
Colorectal Histology	11.77 ± 2.88	11.00 ± 0.00	18.12 ± 0.25	90.34 ± 0.33	89.75 ± 0.19	93.65 ± 0.10	8
Drawing	10.86 ± 1.22	11.00 ± 0.00	11.87 ± 0.93	90.20 ± 0.16	90.00 ± 0.16	89.00 ± 0.16	10
Alien vs Predator	51.48 ± 2.09	28.88 ± 0.00	78.15 ± 0.52	98.52 ± 0.52	98.89 ± 1.42	97.77 ± 0.00	2
COVID-19	20.13 ± 18.66	46.53 ± 0.00	47.22 ± 0.00	93.86 ± 0.16	93.40 ± 0.49	94.56 ± 0.71	3
honey-bee-pollen	49.54 ± 1.30	50.00 ± 0.00	56.94 ± 4.53	93.05 ± 0.00	88.89 ± 0.00	93.55 ± 4.53	2
Speed Limit Signs	30.55 ± 2.27	25.00 ± 0.00	31.48 ± 10.23	83.33 ± 0.00	86.11 ± 0.00	90.74 ± 1.31	4
Japanese Characters	0.03 ± 0.00	0.08 ± 0.00	0.50 ± 0.22	53.17 ± 0.15	62.33 ± 0.16	62.16 ± 0.47 0.45	1566

Figure 2: The Baseline results of diffusion based weight generation, the work that currently overtook the autoencoder approach and is state of the art

4. We have no access to training data but we have a couple of trained deep models that fit very well and we have some ways to traverse the Loss surface and still land on good models without any training needed. [Works like "Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs" and "Essentially no barriers in neural network energy landscape."](#)]

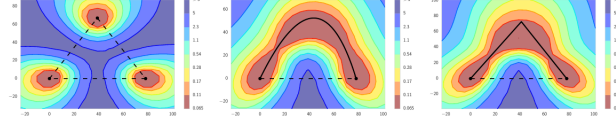


Figure 1: The ℓ_2 -regularized cross-entropy train loss surface of a ResNet-164 on CIFAR-100, as a function of network weights in a two-dimensional subspace. In each panel, the horizontal axis is fixed and is attached to the optima of two independently trained networks. The vertical axis changes between panels as we change planes (defined in the main text). **Left:** Three optima for independently trained networks. **Middle and Right:** A quadratic Bezier curve, and a polygonal chain with one bend, connecting the lower two optima on the left panel along a path of near-constant loss. Notice that in each panel a direct linear path between each mode would incur high loss.

Figure 3: An ensembling method of neural networks allowing the detection of equivalent loss optima through simple polynomial trajectories

5. Same setting as (6) except clients can't send private data or can't provide it in abundance (military images ,hospital XRays ...) so at best the model is accompanied with some pictures of all classes if at all and has varying performance (mid 55%-high 90%) seeing the variety of compute that clients can use. We are tasked with improving the model. reverse engineering to get the input images is prohibited . [Federated Learning is here](#)]

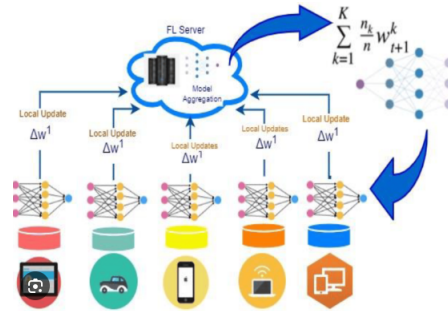


Figure 4: The methodology of Federated learning Client-server updates

6. Clients train their Deep learning model and only send us a trained model (code class and weights) Giving you many samples of the training data that they can't beat .we as Server side model owners are tasked with improving said model with the help of others and sending back a better model or a specific initialization to finetune on their initial training data promising better performance . We analyse the covariance matrix matrices

to detect similarities between weights but have no info on the optimizer , loss landscape , training process .

7. We have access to all the data but the data presents challenging non-linearities . Practitioner and experimental laws are more prevalent . The model is significantly large for the intricacy of it's components for us to know what to focus on and what knowledge was captured on an individual neuron level.

To make sense of the model we track the gradients it leaves behind given specific inputs and infer what works best and what doesn't [Deep Learning maturity]

8. We have access to all the data needed but the data presents singularities/redundancy, Lemma and corollaries and reasonable real world assumptions can be made to approximate the data manifold , the model makes sense and the observable information is sufficient to understand some non-linearities [ML+some Gaussian mixture techniques maturity]
9. We have access to all the data .all theoretical model assumptions met . the model is fully explainable

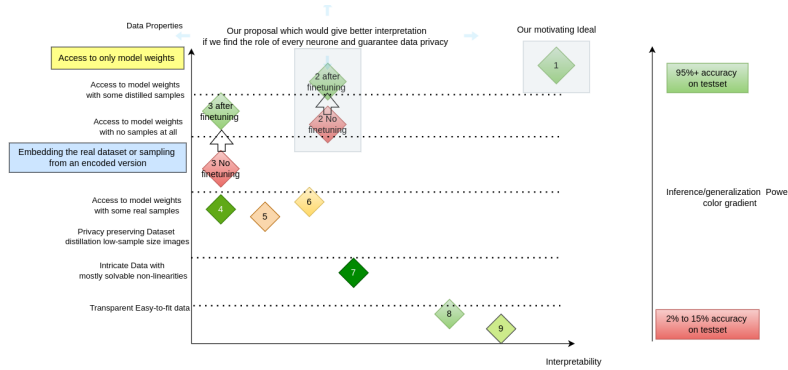


Figure 5: Current relevancy state and Landscape of adjacent works

2.3 Relevance/Impact

Federated Learning is a highly researched area of Deep Learning by Google (<https://github.com/google-research/federated>) and NVIDIA as it's able to change lives and affect real world applications in Healthcare , autonomous driving , speech recognition in Smartphones and IoT Privacy preservation and Efficient computations are the driving motivations to advance the barriers of this domain .

Our impact by investing dedicated architectures at server side to do model aggregation or even perform continual learning tasks between 2 models of differing time period datasets , lies in looking at model parameters as a whole

and arguing about what the model picked up on and how can that expertise be shared with other models .

Through whatever progress we achieve during this PhD proposal we hope to shed more light on the inner workings of Deep Learning and alleviate the fear of trying to understand a complex structure by breaking it down to multiple bite-sized interpretable segments that we can trace back the success or failure of clearly as well as understand more of the properties of training and how to enforce them on smaller models using less compute .

3 Proposed Methodology

These works that seek to do reconstruction operate in a self-supervised/unsupervised manner where as we provide a target with desirable properties and even if we reach equivalent models , what matters most is knowing what guided that deviation to better make the union operand . the transformer architecture has occupied the position of state of the art go to except for some message passing , time series , audio manipulation tasks for the past few years and we motivate the choice here by these works that learn mathematical operands ["Grokking : generalization beyond over-fitting on small algorithmic datasets "]and ["omni-grok:grokking beyond algorithmic data"]which is our main goal but they also come with embeddings that can decode to the weight space unlike some dimensionality reduction techniques that aren't reversible .

In practice ,We have no image Data , At best a distilled version of each class (for scale:60k MNIST training images replaced by 5 images per class) .Experimentally to simulate this ,we create a model zoo of all possible subsets of classes in MNIST and we have 2 flattened weight sets and we aim to combine their knowledge using a double-input transformer .

We hope that by analyzing the Gradients left like (7) or detecting patterns in the auto-encoder's attention heatmaps or analyzing the Topological properties of the Predicted parameters against the ground truth ones to bridge the gap between the the weak performance predicted weights and the ground truth target weights or an equivalent extrapolated like (4) .

the finetuning in (3) takes a couple hundred steps (essentially a single epoch) to reach desirable performance but breaches Federated Learning concerns. distilled data takes less steps (because the number of samples is less) to achieve the same reconstructed performance.

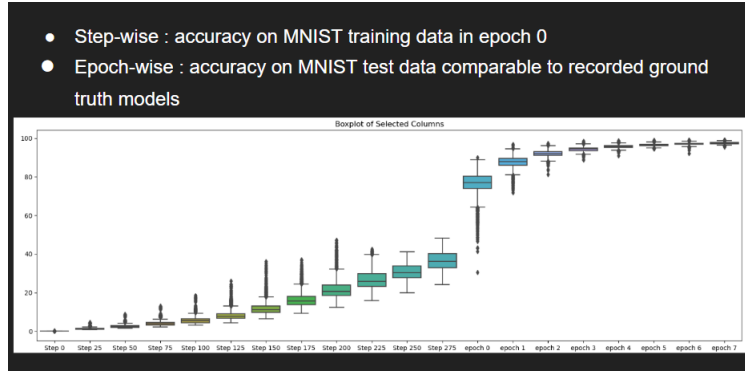


Figure 6: Example of the finetuned model predicted weights these weights learn faster than normal initialization and have more potential to catchup to the original target networks

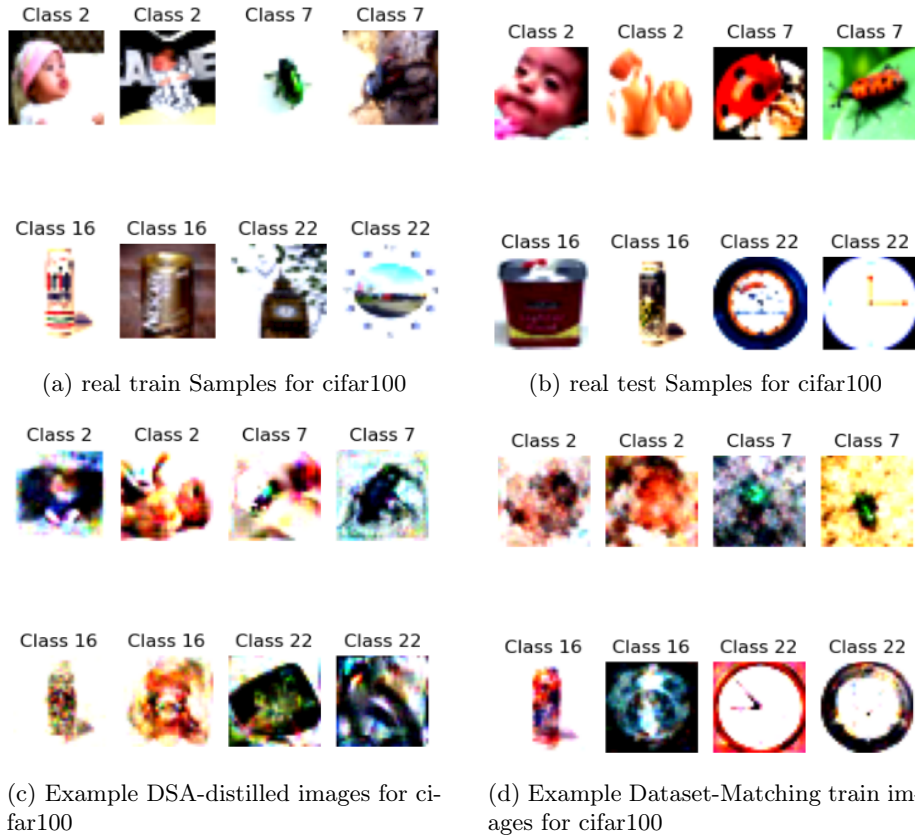


Figure 7: On the privacy preserving aspect of Distilled data

We simplify the task of predicting weights by imposing a simple 3-convolution and 2-dense layers architecture with small percentage of dropout .This is an intermediate step because once we prove the effectiveness of our method for knowledge combination and doing regression on high performing weights we can begin to extend this by using NASBOT or the OTMANN distance mentioned in ["Neural Architecture Search with Bayesian Optimisation and Optimal Transport"]

	conv3	conv5	max-pool	avg-pool	fc
conv3	0	0.2	∞	∞	∞
conv5	0.2	0	∞	∞	∞
max-pool	∞	∞	0	0.25	∞
avg-pool	∞	∞	0.25	0	∞
fc	∞	∞	∞	∞	0

Table 1: An example label mismatch cost matrix M . There is zero cost for matching identical layers, < 1 cost for similar layers, and infinite cost for disparate layers.

Figure 8: the OTMANN inter-layer distance later used for NASBOT to quantify the distance between different neural network architectures

The work in ["Geometric Dataset Distances via Optimal Transport" by Kirthevasan Kandasamy et al] finds a distance formulation on the normal image manifold that allows given 2 datasets the effective generalization gap fall off when training on one and Transfer Learning to the other, their main find that could be of use to us is correlating the performance drop with how larger distances .

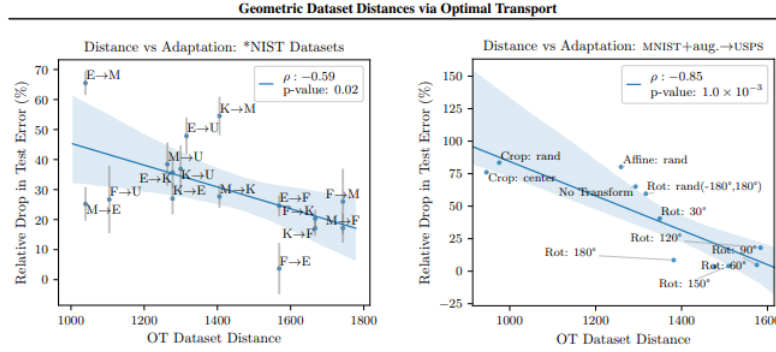


Figure 5. Dataset distance vs. adaptation for *NIST datasets (M: MNIST, E: EMNIST, K: KMNIST, F: FASHION-MNIST, U: USPS). The error bars correspond to ± 1 s.d. over 10 repetitions.

Figure 6. Dataset distance vs. adaptation between MNIST with various transformations applied to it and USPS. While cropping the MNIST digits leads to better adaptation, rotating them degrades it, both in agreement with the dataset distance.

Figure 9: The linear correlation between model performance and the author's proposed dataset distance , allowing an estimate of finetuning accuracy loss if Transfer learning were to take place in advance of training

The work in ["Manifold Matching via Deep Metric Learning for Generative Modeling" by Mengyu Dai et al] is a recent work that looks at Geodisc distances as a way to capture the underlying information that other distances fail to quantify . they base their comparisons on the original infoGAN latent space

manipulation paper ["Unsupervised Representation Learning with deep convolutional generative adversarial networks" by Alec Radford et al] in which they are able to interpolate, perform the "addition" operator on latent representations of images that make no sense when superposed on the image space. The result is embeddings that decode to perfectly clear and meaningful images. We mention this because it might motivate changes in the way our architecture is built or chosen and allow us to mimic the property that let this fusion in embeddings be so successful as we try to extend it to weights and biases.

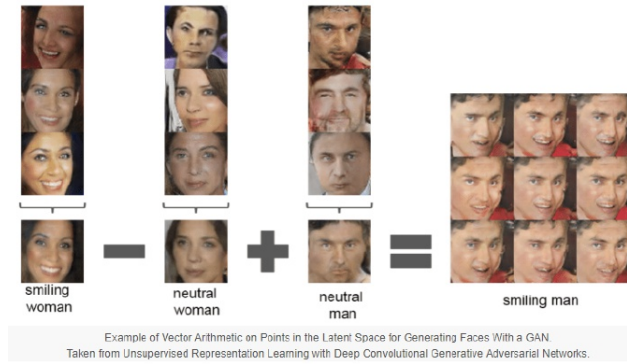


Figure 10: Example of element wise operations on the latent space of infoGAN and successful semantical class fusion

["The Representation Theory of Neural Networks" by Marco Antonio Armenta represents neural networks as Quivers/graphs] hence motivating the usage of Graph distance on the Mapper Algorithm's mappings (a search for a good filtration function is needed, Fischer information metric is a candidate (?), type/shape of cover, percentage of overlapping coverage) to capture the manifold's properties. (4) maybe has found isomorphisms between weights space and loss surface (?)

Additionally, other works show that only 5% of the weights are needed and pruning/sparsification techniques can be used. ["The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks"]

["Grokfast: Accelerated Grokking by Amplifying Slow Gradients"] proposes a low-band pass filter in the physics spectral sense and sees the weights with "slow" varying gradients responsible for grokking and eventually generalization and "fast" varying ones as needed as a short term memory and needed for specific tasks. This distinction drives a motivation to find what is a slow-fast 5% weight Combination we want our inferred models to have.

Less formally and more algebraically incentivised: even if we find a desirable property limited to some weights. The fact that theories exist allowing the passage between Quiver Theory and Group theory mean that if we find an algebraic Ideal (an absorbing subgroup) we would search for the operation to apply between these desirable weights and the rest of the neural network. Once said

2 Amplifying the Low-Frequencies of the Stochastic Gradients

2.1 Filter Design

Amplifying the low-frequencies of the gradients $g(t)$ can be achieved by adding a low-pass filtered signal $g(t)$ to itself. Let $h(t)$ be a discrete-time low-pass filter (LPF) defined over the training iteration t . For simplicity, we assume a univariate time-invariant low-pass filter $h(t)$ uniformly applied across every model parameter θ . Using a convolution operator $*$, we denote the modified gradient $\hat{g}(t)$ as:

$$\hat{g}(t) = g(t) + h(t) * g(t), \quad (3)$$

which can then be plugged into the parameter update function u of the optimizer:

$$\hat{u}(t) = u(\hat{g}(t), t) = u(g(t) + h(t) * g(t), t). \quad (4)$$

In the dual domain, equation (3) is equivalent to:

$$\hat{G}(\omega) = G(\omega) + H(\omega)G(\omega) = (1 + H(\omega))G(\omega), \quad (5)$$

where $H(\omega) = \sum_{t=0}^T h(t)e^{-i\omega t}$ is the transfer function of the filter $h(t)$. Our goal can therefore be restated as to design a filter $h(t)$ with low-pass characteristics in its transfer function $H(\omega)$.

For the demonstration of our initial claim, we first take the simplest strategy: the LPF $h(t)$ is a windowed moving average (MA) over a fixed-size window of width w .

$$h(t) = \frac{\lambda}{w} \Pi\left(\frac{t}{w} - \frac{1}{2}\right) = \begin{cases} \lambda/w, & \text{if } 0 \leq t < w. \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The function Π stands for the Heaviside Pi function, which has value of one in the interval $[-0.5, 0.5]$ and of zero elsewhere. This filter has only two scalar hyperparameters: the scalar factor λ and the window size w . As shown in Algorithm 1, we implement this filter h with a fixed-capacity queue Q storing the intermediate parameter updates $u(t)$ into the queue Q . The average of the queue Q is the low-pass filtered gradients which is added to the current parameter update at each optimizer step.

Figure 11: The author of Grokfast finds a way to distinguish the generalizing neurons from the more adaptable ones via studying the fourrier transform of their gradients and applying a low pass filter

idea is found the Quiver equivalent would need to be expressed in the neural network space.

4 Research Plan

4.1 Architecture and steps

I outline the suggestions that I think best befit my current progress and explain them in greater detail in an annexed document of the proposal along with code to replicate my current work and my splitMNIST model zoo at <https://github.com/AymenTlili131/Federated-Continual-learning-> :

- Dataset Zoo improvements could be made as my current work was limited to target models of a certain fixed architecture that satisfied criteria both in performance , duration of training and generalization power on their respective tests . The question of having to combine models of differing sizes/architectures is of natural occurrence and we direct the prospects of this on optimal transport , NASBOT and Quiver representation .
- Changes to the chosen architecture , seeing that the learning is not the outcome of a single behaviour but the stacking and emergent behaviour

of many , a choice could be made on how to represent models in the zoo .Perhaps , better as a Graph or a higher order simplex

- Interpretability tools that help us , which Autoencoder input features are causing the most gradients and faillures . In this case the features are CNN weights
- Better Losses and objective definition that take into account the topological properties of normal ground truth trained models

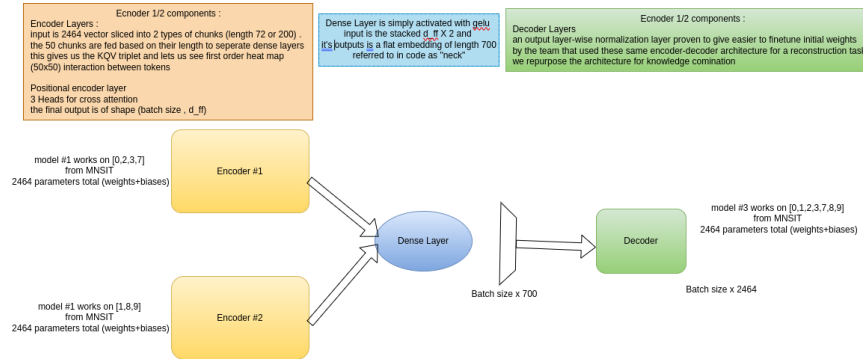


Figure 12: The architecture I propose for combining model parameters based on previous works that would do reconstruction instead and sample from the neck's latent space

4.2 Current Observations and leads

The personal progress that I have made allows me to infer the following :

- We have tried using checkpoints from different points of time , and what we uncovered is that models that come from Autoencoder checkpoints with high training loss are much harder to finetune seeing it takes many epochs beyond the first 10-15 that we give for finetuning to leave the 0-20% accuracy area on the combined task .
- the decoder is the part of the model that's struggling to learn . as we observed the attention heatmaps being learned by the 2 encoders and decoder : normal patters of starting out as random values akin to noise. The Encoders become uniform as the model loss drops significantly and stays there for close to 100-130K steps [80 autoencoder epochs] before the attention maps start showing patterns and relations among the sliced token weights (we tokenize the flattened weights into a fixed maximum length of 50 and let 2 types of dense layers operate on them)and those patterns vary steadily as training continues .The decoder starts off the same but even after much longer training the uniform heatmaps persist and no patterns aside from a single dominating token are attended to .To be noted the

architecture is the same as konstantin's (section 2.2 (3)) that advocated for normalizing outputs around each layer's mean and penalizing deviation from layer-wise distribution by adding a regularization term to his contrastive and MSE loss functions except we doubled the number of encoders and thus the model's inputs. the dense layer connecting the outputs of the encoders and the decoder showed that it helps as an ablation study demonstrated that it's absence delays the formation of even the uniform pattern in heatmaps .

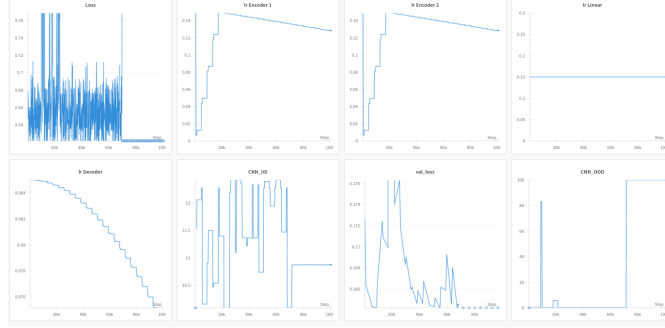
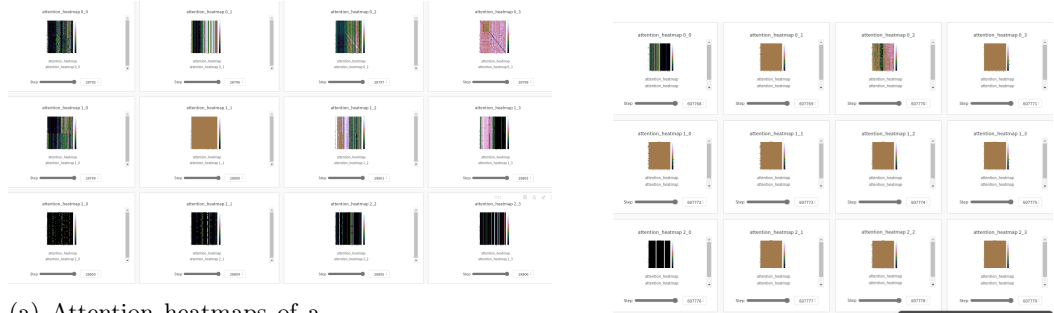


Figure 13: Example of Transformer Training and the need for separete schedulers for each group of weights , specifically the decoder



(a) Attention heatmaps of a model trained on the Fisher information metric for the validation loss to stabilize (90 epochs)

(b) Example of attention heatmaps of a model trained with the MSE+layer wise loss normalization cost function

Figure 14: we track the interactions of 50 tokens among each other and their activations. 3 rows for each component is displayed : 2 encoders 1 decoder. Each compent has 4 Heads represented in each column. Using the cubehelix color palette we notice a huge difference of the patterns learned and more starkly the slow formation of said patterns for the decoder when training [the brown uniform maps]

c) The mapper algorithm confirms that training forces weights to branch out

and bifurcation moments in the training process cause an outburst in the norm of a neurone weights and evidently the performance of the overall model

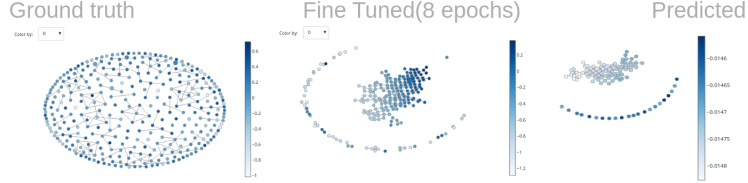


Figure 15: We apply the mapper algorithm weights predicted by the autoencoder (right) , same said weights after finetuning(middle) and the target ground truth models training on the same data but normally(10epochs) , using a cubical cover , 15% interleaving percentage and DBSCAN with minimum number for a cluster to form being 20. Notice the stacked projection function values for the predicted weights and how it expands as training occurs arriving at a much larger range of values (confirming the "topology of learning").Our PhD's proposal is to bridge the gap between the right image and left one.

This has already been explored in the works["Topology of Learning in Artificial Neural Networks" by Maxime Gabella]and the ["TopoAct: Visually Exploring the Shape of Activations in Deep Learning" by Archit Rathore et al.]but the actual reason that causes the branching remains unknown .

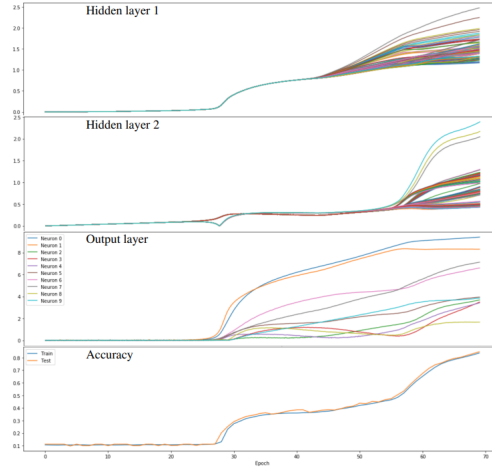


Figure 8: *Top*: Norms of the weights per neuron for the two hidden layers and the output layer. The uniform increase is triggered by branching in the output layer. The branching is followed by a short period of reshuffling, before a converging period. *Bottom*: The accuracy increases in jumps: 10% (chance), 40%, 80% (with more training steps, this would be followed by a slow increase towards 95%).

Figure 16: Tracking the spiking of the norm of weights and the respective rise of accuracy during training

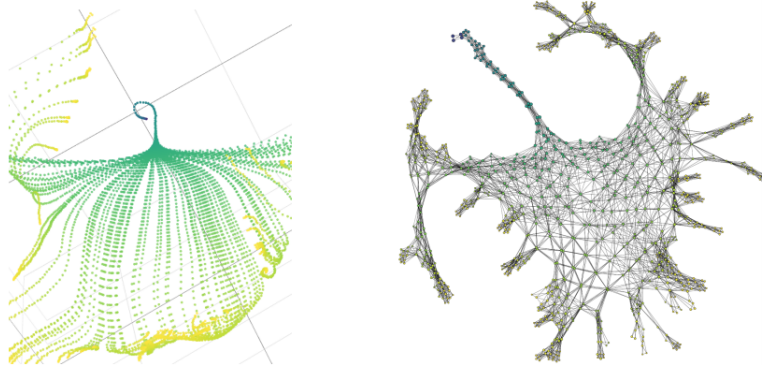


Figure 6: *Left*: Detail of the PCA projection of the evolution of the weights for the first hidden layer, as in the top left of Figure 5. Several phases are visible: uniform evolution (blue), parallel evolution along a surface (green), chaotic evolution (yellow). *Right*: learning graph representing the surface as a densely connected grid (filter function given by the first 3 PCA directions).

Figure 17: Colored from blue to yellow, 'Topology of learning' author uncovers the different directions of branching started from a random initialization where values are close and using the mapper algorithm to connect underlying stages on the MNIST dataset training

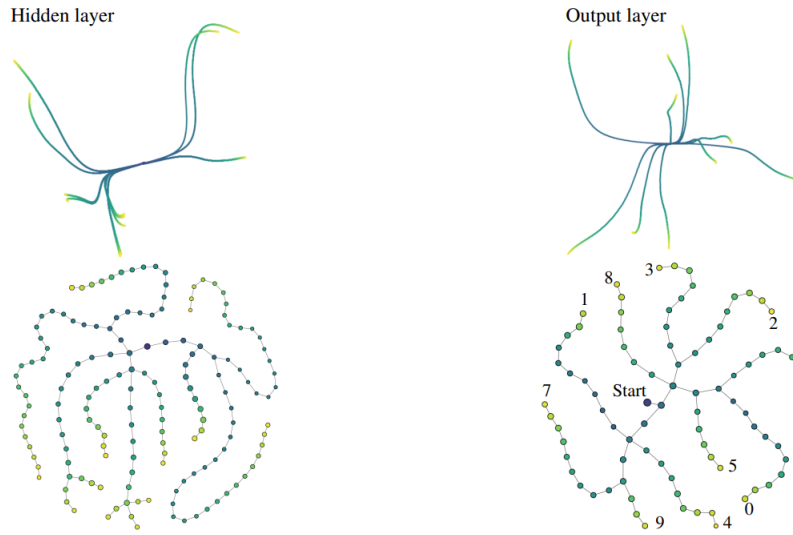


Figure 3: Evolution of weights for a neural network with one hidden layer, initialized at zero (colored by training step). *Top*: PCA projections. *Bottom*: Learning graphs. *Left*: Hidden layer. *Right*: Output layer (digits are indicated).

Figure 18: mapping the neurone evolution with the according predicted classes

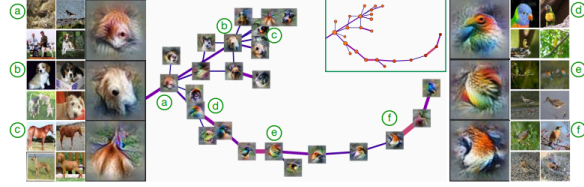


Figure 6: Bird-mammal bifurcation. Configuration: layer 5a, Euclidean norm, 70 intervals, 30% overlap, fixed ϵ for DBSCAN.

Figure 19: Topoact and the power of the Mapper algorithm to segment structures through branching shapes/meanings

- d) we confirm the correlation found in ["Contractive Auto-Encoders: Explicit Invariance During Feature Extraction" by Salah Rifai] between the forbenius norm of the jacobian Autoencoder weights(lower is better and might signal loss flatness that has been shown to help generalization) and it's loss (lower is better) and the results show also that the values predicted are "under-saturated" meaning that they , if activated by a sigmoid function for instance , would fall under 5% which gets assigned an activation of 0 whereas the paper mentions that "over-saturation" (being above 95% before getting activated is the desired property of generalizing models .

	Model	Test error	Average $\ J_f(x)\ _F$	SAT
MNIST	CAE	1.14	$0.73 \cdot 10^{-4}$	86.36%
	DAE-g	1.18	$0.86 \cdot 10^{-4}$	17.77%
	RBM-binary	1.30	$2.50 \cdot 10^{-4}$	78.59%
	DAE-b	1.57	$7.87 \cdot 10^{-4}$	68.19%
	AE+wd	1.68	$5.00 \cdot 10^{-4}$	12.97%
	AE	1.78	$17.5 \cdot 10^{-4}$	49.90%
CIFAR-10	CAE	47.86	$2.40 \cdot 10^{-5}$	85.65%
	DAE-b	49.03	$4.85 \cdot 10^{-5}$	80.66%
	DAE-g	54.81	$4.94 \cdot 10^{-5}$	19.90%
	AE+wd	55.03	$34.9 \cdot 10^{-5}$	23.04%
	AE	55.47	$44.9 \cdot 10^{-5}$	22.57%

Figure 20: The author presenting a type of Autoencoder that has sparse highly saturated weight matrices where the activated values are enough to decrease error regardless of noisy input and this generalization power correlates with the norm of the gradients of neurons

- e) The mapper algorithm representation of weight distribution confirms that models trained normally cover a more uniform space and have intricacies that could relate them with the right filtration function .Whereas the finetuned ones occupy a subspace and still give a similar performance on the test set even if the finetuning lasted 1 epoch . meaning that further training helps learn more complex structures . The topological evolu-

tion of such said structured by the model is illustrated in ["Exposition and Interpretation of the Topology of Neural Networks" by Rickard Br  l Gabrielsson and Gunnar Carlsson "]

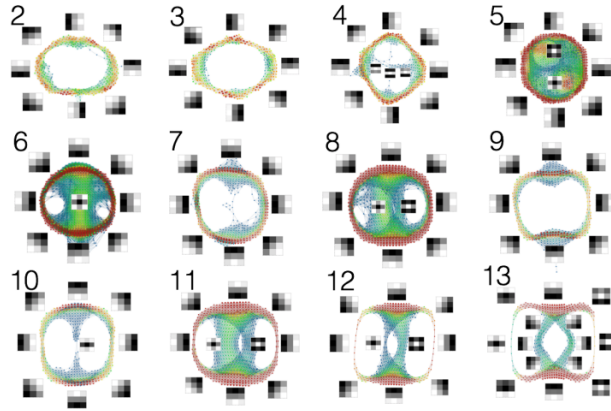


Fig. 10: Mapper applied to the convolutional spatial filters of VGG16

Figure 21: The author tracks the structures learned by a VGG16 model through applying Mapper to the weights of different layers to unveil the form taken

- f) the MSE of a predicted models' weights and those from post-finetuning indicate that the finetuning DOES NOT take us to to our target but an equivalent network .but the Wasserstein distance is the only one among MSE , LWLN , Contractive loss , KL divergence to decrease meaningfully as finetuning progressed . Suggesting that (4) could help us "jump" between local optima .

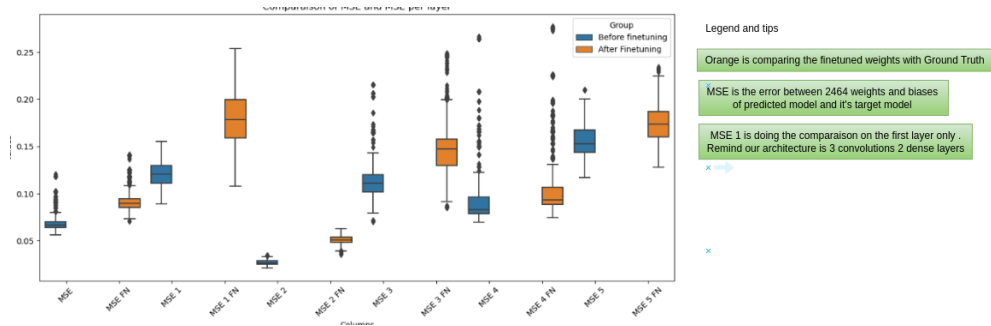


Figure 22: Boxplot of 3400+ pairs of My autoencoder's predicted weights and distance from ground truth before and after Finetuning. I present such reseults for each layer of my 3 convolution 2 danse layered CNN zoo suggesting from the increase of error after finetuningthat the finetuned model is different from the ground truth one and that MSE is not the proper metric for handling the optimization , WS is proven in a separete document to be a better candidate

5 Resources

I will outline , from experience what I think would work for me best . If you happen to see the value of the research i'm doing. You have complete liberty to disregard these inquiries if they are out of your reach or you don't see them as necessary .

Human Ressources			
Person	Minimal requirement	Ideal setting	Passable Setting
Advisor	Someone with Topology and Compter science knowledge willing to listen to new ideas and crazy approachs and give an honest opinion with no suger-coating their feelings about my work.Responding to an email with one or in-person within atleast 3 working days. Willingless to grow and learn if research leads us into new domains.	weekly/biweekly meeting : If i happen to send 1-2 papers beforehand please read them so that an 1hour-long meeting spent explaining the papers becomes a 15-25 minute checkup on what the best next steps are and we spend it designing convincing experiments and getting guidance and discussing results	help provide any form of hardware ressource .Will sit down and try to read my code if i hit a wall for a prolonged period and provide a minimalistic notebook version to replicate a problem or a theoretical inconsistency. Awareness of minimal mental health needs(breaks to avoid burnout)
Collaborating PhD student (1-3 people)	familiarity with Github,Pytorch(c or pytorch implementation) the frameworks we'd be using and some readiness to partake in writing Latex formatted documents and copying package virtual environements	Background in persistant homology or Optimal transport or NAS (NASBOT is coded primarily in Fortan) or ODE solvers out of the box (Julia,MatLab) . empathetic,humble, patient and forgiving	Willingless to learn and minimal fear of the unknown and strong communication skills

Material Ressources			
Stage of work	Minimal require- ment	Ideal setting	Passable Setting
Zoo (re)-training [if needed]	a personal/shared work-pc with atleast 200 GB hard drive stor- age and cuda compatible GPU	1to3 remotely accessible Servers/workspaces with RTX A5000 or any ray-tracing capable GPU with 16-24 GB or more VRAM and 2 TB of Harddrive storage	RTX 3080 or above and a hardrive with a storage partition of 500GB
Model architecture smart Grid searches [guided by optuna Package]	same as above with 8 i7 or more cores	10\$ premium account for Wandb.com to track experiments progress and save the best models for finetuning testing	—
Custom dataset Dataloading	pytorch/tensorflow and adequate pan- das/polars virtual environment [pip install rights and conda install rights not blocked by firewarll or remote server session]	16GB or above normal RAMand- VRAM and 2 133 MHz or 3 200 MHz	—
Model Training	same as above	if grid search decides we need bigger models 24GB VRAM would help , High normal RAM as well if we will work with NetworkX graphs	16Gb RAM/12Gb VRAM
model testing and Zoo CNN finetun- ing	Raytracing helps alot here , any RTX card to do Forward passes or A5000 or above equiva- lent. no training is needed and thus we can work with as little batch size as needed (memory is not an issue) 19	(depends on chosen model and metrics to compute) —	—
Living	a mattress,access to WC,220V- electric- ity and running wa- ter	a stipend / scholar- ship grant	cafeteria ac- cess/dorm/shared renting for the duration of the phD

The current state of research was reached through a personal laptop with 1TB SSD harddrive and 16GB RAM and a GPU of MX130 with 2GB of VRAM. The digital research center of sfax, Tunisia has provided me with an RTX3060 GPU i9 500GB workstation and while I was training my model zoo with remote access(FTP and ssh) to 2 servers having TeslaT4 GPUs. I eventually acquired my own 2TB HDD drive to store checkpoints . A coworker did occasionally contribute on weekends by letting me train some larger transformer models on an RTX3090 24GB GPU.

My advisor was not willing to hear me explain what topology is and until the diffusion based generation paper came out I was being harrassed for code quality as they thought the transformer could no produce such bad results (before finetuning) .

I only got to test out training with wasserstein Loss while my former advisor was on vacation and semi gave up on the project and let me do whatever I wanted. I am grateful to them all but my contract with this lab will not be renewed and I seek to start a new chapter with you esteemed advisors in this project or if you see I'm a better fit for something else. I have some RL related projects using blender, unity and ml-agents package which are not published on my portfolio as well as my senior year transcripts which will be committed to github by the end of the week . Feel free to contact me with your proposals .

Portfolio site: aymentlili131.github.io

Email: Aymentlili131@gmail.com